Lab 10

PL/pgSQL: Anonymous Blocks

CSE 4308

DATABASE MANAGEMENT SYSTEMS LAB

Prepared by:
Samnun Azfar, Jr. Lecturer, CSE
Samia Nawsheen, Jr. Lecturer, CSE
Academic Year: AY25-26

Contents

1 Lab Workflow **2**

2 Database Schema & Data **4**

3 Introduction to PL/pgSQL **6**
3.1 Anonymous Blocks 6
3.2 Outputting Text 6

4 Variables and Arithmetic **6**

5 Single Row Queries (SELECT INTO) **6**

6 Control Structures **8**
6.1 Conditional Logic (IF/ELSE) 8
6.2 CASE Statement 8

7 Loops **8**
7.1 WHILE Loop 8
7.2 FOR Loop (Iterating Query Results) 9

8 Using WITH (CTEs) in Blocks **10**

9 Lab Tasks **11**

1 Lab Workflow

Repeat this subsection for every single lab.

1. Connect to the internet by running nmtui in terminal, add a dsl connection, then go back and activate that dsl connection. Make sure that your net id is not being used in any other pc or router, if you face any issue, use any other net id from your friends which is not being used.

2. Download all the files posted in the google classroom.

3. Disconnect from the **dsl** connection via nmtui, connect to the ethernet/wired connection via nmtui. If no such connection exists, create an **ethernet** connection and then activate.

4. **Create Your Role and Database**

   Inside the PostgreSQL shell, create your own role (user) and database.

   1. In the Linux terminal:

```
psql
```

   Expected prompt:

```
cse=#
```

   2. Create Your Role and Database

   Copy and run the following commands inside PostgreSQL:

```
-- 1. Terminate existing database connections
SELECT pg_terminate_backend(pid)
FROM pg_stat_activity
WHERE datname = 'cse4308_12345'
  AND pid <> pg_backend_pid();

-- 2. Remove existing database
DROP DATABASE IF EXISTS cse4308_12345;

-- 3. Remove existing role
DROP ROLE IF EXISTS student_12345;

-- 4. Create role
CREATE ROLE student_12345 LOGIN;

-- 5. Set password
ALTER ROLE student_12345
WITH PASSWORD 'student12345';

-- 6. Create database
CREATE DATABASE cse4308_12345
OWNER student_12345;

-- 7. Grant privileges
GRANT ALL PRIVILEGES
ON DATABASE cse4308_12345
TO student_12345;
```

3. Exit PostgreSQL:

```
\q
```

5. **Connect Using Your Newly Created User**

   1. Connect to Your Database:

   Use the following command to log in as the `student_12345` user:

```
psql -h localhost -U student_12345 -d cse4308_12345
```

   2. Verify the Connection:

   Verify that you are connected to the correct database with the following command:

```
\conninfo
```

   Expected output:

```
Database: cse4308_12345, User: student_12345
```

   3. Exit PostgreSQL:

```
\q
```

6. **Load the Lab SQL File (MUST be run outside psql)**

   Load the SQL file that sets up the tables and inserts initial data.

   1. Load the SQL File:

   Run the following command to load the lab SQL file:

```
psql -h localhost -U student_12345 -d cse4308_12345 -f /home/cse/Downloads/
 lab_10_setup.sql
```

   This will: - Drop old tables (if they exist). - Create new tables. - Insert initial rows of data.

7. **Reconnect and Verify Tables**

   After loading the SQL file, reconnect to PostgreSQL to verify that the tables have been created.

   1. Reconnect to Your Database:

```
psql -h localhost -U student_12345 -d cse4308_12345
```

   2. Check Tables:

   Verify the tables have been created by listing them:

```
\dt
```

3. View Table Data:

Verify the data by running the following SQL queries:

```
SELECT * FROM <table_name>;
```

8. Open a text editor and save it in the appropriate submission name format `fullid_CSE4308_L10_T1.sql`. You will use this text editor to write your solutions, make sure to save it after writing each solution. The structure of this file will be:

```
-- Task 1
INSERT INTO students (student_id, name, major, gpa)
VALUES (104, 'David', 'ME', 3.60);
--Task 2
SELECT name, gpa FROM students WHERE major = 'CSE';
```

==========

2 Database Schema & Data

Instructions: Save this code as `lab_10_setup.sql` and run it using `\i lab_10_setup.sql` in your psql terminal. This script generates a dataset of 1,000 employees with varying salaries and departments to simulate a real-world scenario.

```
-- 1. CLEANUP
DROP TABLE IF EXISTS employees;

-- 2. CREATE TABLE
CREATE TABLE employees (
    employee_id SERIAL PRIMARY KEY,
    first_name  VARCHAR(50),
    last_name   VARCHAR(50),
    salary      NUMERIC(10, 2),
    department  VARCHAR(50)
);

-- 3. INSERT KEY PERSONNEL (Deterministic Data)
-- We insert these specific people so we can always find them in queries
INSERT INTO employees (first_name, last_name, salary, department) VALUES
('Alice', 'Director', 25000.00, 'Management'),
('Bob', 'Architect', 15000.00, 'IT'),
('Charlie', 'Manager', 12000.00, 'Marketing'),
('Rahim', 'Uddin', 4500.00, 'Operations');

-- 4. BULK INSERT (Randomized Data)
-- This generates 1000 fake employees
INSERT INTO employees (first_name, last_name, salary, department)
SELECT
    -- Pick a random first name from the array (Mixed International & Bangladeshi)
    (ARRAY[
        'John','Jane','Mike','Sara','Tom','Emily','Chris','Anna','Steve','Laura',
```

```
        'Rahim','Karim','Fatima','Ayesha','Rafiq','Jabbar','Nasrin','Monir','Sultana',
    'Kamal'
    ])[floor(random() * 20 + 1)],

    -- Pick a random last name (Mixed International & Bangladeshi)
    (ARRAY[
        'Smith','Johnson','Williams','Jones','Brown','Davis','Miller','Wilson','Moore'
    ,'Taylor',
        'Ahmed','Hossain','Khan','Chowdhury','Rahman','Islam','Sarker','Uddin','Ali','
    Hasan'
    ])[floor(random() * 20 + 1)],

    -- Generate random salary between 3,000 and 18,000
    ROUND((random() * 15000 + 3000)::numeric, 2),

    -- Pick a random department
    (ARRAY['IT', 'HR', 'Management', 'Marketing', 'Sales', 'Operations'])[floor(random
    () * 6 + 1)]
FROM generate_series(1, 1000);

-- 5. VERIFY
-- Should return 1004 rows
SELECT count(*) as total_employees FROM employees;
```

Listing 1: lab_10_setup.sql

3 Introduction to PL/pgSQL

PL/pgSQL (Procedural Language/PostgreSQL Structured Query Language) extends SQL by adding procedural constructs. While SQL is declarative (stating *what* you want), PL/pgSQL allows you to state *how* to achieve it using loops, variables, and logic.

3.1 Anonymous Blocks

An **Anonymous Block** is a piece of code that is executed immediately and not stored in the database. In PostgreSQL, this is done using the `DO` statement.

Structure: The code body must be enclosed in Dollar Quotes (`$$`).

```
DO $$
DECLARE
    -- Variable declarations
BEGIN
    -- Execution logic
EXCEPTION
    -- Error handling
END $$;
```

3.2 Outputting Text

PostgreSQL uses `RAISE NOTICE` to print messages to the console (similar to `System.out.println` in Java or `printf` in C).

4 Variables and Arithmetic

Variables must be declared in the `DECLARE` section. You can initialize them using the `:=` operator.

```
DO $$
DECLARE
    v_radius    NUMERIC := 5.5;
    v_pi        CONSTANT NUMERIC := 3.14159;
    v_area      NUMERIC;
BEGIN
    -- Formula: Area = pi * r^2
    v_area := v_pi * (v_radius ^ 2);

    RAISE NOTICE 'Radius: %', v_radius;
    RAISE NOTICE 'Calculated Area: %', v_area;
END $$;
```

Listing 2: Geometry Calculation

5 Single Row Queries (SELECT INTO)

To use data from a table inside a PL/pgSQL block, you cannot simply write `SELECT * FROM table`. The result must be stored in a variable using the `INTO` keyword.

Rule: The query must return exactly **one** row. If it returns zero rows or multiple rows, it may raise an error or warning depending on configuration.

```
DO $$
DECLARE
    v_emp_id        INT := 1;
    v_full_name     TEXT;
    v_salary        NUMERIC;
BEGIN
    -- 1. Fetch data into variables
    SELECT first_name || ' ' || last_name, salary
    INTO v_full_name, v_salary
    FROM employees
    WHERE employee_id = v_emp_id;

    RAISE NOTICE 'Employee: % earns %', v_full_name, v_salary;

    -- 2. Use variables in DML (Update)
    UPDATE employees
    SET salary = salary + 100
    WHERE employee_id = v_emp_id;

    RAISE NOTICE 'Salary updated.';
END $$;
```

Listing 3: Fetching Employee Data

6 Control Structures

6.1 Conditional Logic (IF/ELSE)

PL/pgSQL supports `IF-THEN-ELSIF-ELSE` logic.

```
DO $$
DECLARE
    v_salary NUMERIC;
BEGIN
    -- Get Alice's salary
    SELECT salary INTO v_salary FROM employees WHERE first_name = 'Alice';

    IF v_salary > 10000 THEN
        RAISE NOTICE 'Status: Executive Level';
    ELSIF v_salary BETWEEN 5000 AND 10000 THEN
        RAISE NOTICE 'Status: Manager Level';
    ELSE
        RAISE NOTICE 'Status: Entry Level';
    END IF;
END $$;
```

Listing 4: Salary Evaluation

6.2 CASE Statement

The `CASE` statement is useful for selecting a value based on multiple patterns.

```
DO $$
DECLARE
    v_grade CHAR(1) := 'B';
    v_result TEXT;
BEGIN
    CASE v_grade
        WHEN 'A' THEN v_result := 'Excellent';
        WHEN 'B' THEN v_result := 'Good';
        ELSE v_result := 'Average';
    END CASE;

    RAISE NOTICE 'Grade B means: %', v_result;
END $$;
```

7 Loops

7.1 WHILE Loop

Executes as long as the condition is true.

```
DO $$
DECLARE
    v_counter INT := 1;
BEGIN
    WHILE v_counter <= 3 LOOP
        RAISE NOTICE 'Count: %', v_counter;
        v_counter := v_counter + 1;
```

```
    END LOOP;
END $$;
```

7.2 FOR Loop (Iterating Query Results)

This is the most powerful loop in PL/pgSQL. It allows you to iterate over every row returned by a query without explicitly opening or closing cursors.

```
DO $$
DECLARE
    r RECORD; -- A generic holder for row data
BEGIN
    RAISE NOTICE '--- IT Department Staff ---';

    -- "r" becomes a row for every iteration
    FOR r IN
        SELECT first_name, salary FROM employees WHERE department = 'IT'
    LOOP
        RAISE NOTICE 'Name: %, Salary: %', r.first_name, r.salary;
    END LOOP;
END $$;
```

Listing 5: Looping through rows

8 Using WITH (CTEs) in Blocks

You cannot start a `DO` block with `WITH`. However, you can use Common Table Expressions (CTEs) inside the SQL statements contained within the block.

```
DO $$
DECLARE
    r RECORD;
BEGIN
    -- We want to find employees who earn more than the average
    FOR r IN
        WITH AvgSal AS (
            SELECT AVG(salary) as val FROM employees
        )
        SELECT e.first_name, e.salary, a.val as average
        FROM employees e, AvgSal a
        WHERE e.salary > a.val
    LOOP
        RAISE NOTICE '% earns % (Above Avg: %)',
            r.first_name, r.salary, ROUND(r.average, 2);
    END LOOP;
END $$;
```

Listing 6: CTE inside Anonymous Block

9 Lab Tasks

Scenario: You are the database administrator for a startup. You need to perform complex updates and generate reports using PL/pgSQL.

Task 1

Create 10 dummy `'Test User'` accounts.

- Use a loop to insert 10 rows.
- First Name: `'User'`, Last Name: Loop Counter.
- Salary: 1000 * Loop Counter.
- Department: `'Testing'`.

Task 2

Count the number of employees in the `'IT'` department *without* using the standard `COUNT(*)` aggregate function.

- Initialize a counter variable to 0.
- Loop through the employees in `'IT'`.
- Increment the counter by 1 for every row found.
- Print: "Manual count found [X] employees in IT".

Task 3

Round all salaries to the nearest 100.

- Iterate through all employees.
- Use the `ROUND()` function with a negative scale (e.g., -2 rounds to nearest 100).
- Update the salary.
- Print: "Rounded salary for [Name] to [New Salary]".

Task 4

Redistribute wealth.

- Identify the richest employee (Order Desc, Limit 1).
- Identify the poorest employee (Order Asc, Limit 1).
- Deduct 1000 from the richest and add 1000 to the poorest.
- Print the names of both involved parties.